

German Traffic Sign Recognition using Deep Convolutional Neural Networks

Abstract

In this study, a tailored Convolutional Neural Network is used on German Traffic Sign Recognition Benchmark (GTSRB). The model attained an accuracy of 93.86 per cent, which shows it to be an efficient learner of features. The lightweight architecture demonstrates good potential for real-time implementation in embedded autonomous driving systems, although minimal preprocessing and no data augmentation were employed.

Keywords – Traffic Sign Recognition, Convolutional Neural Network (CNN), GTSRB, Image Classification, Deep Learning, Embedded Systems, Autonomous Driving.

1. Introduction

‘Traffic sign recognition (TSR)’ has turned out to be an important application in the field of ‘computer vision and intelligent’ transportation systems. It is essential in the development of autonomous driving technologies and higher ‘driver assistance systems (ADAS),’ where real-time identification and categorization of road signs are essential to the safety of the road and adherence to road regulations.

Wrong category or inability to recognize traffic signs may result in dangerous driving actions, and TSR is a traffic-related task of high importance in terms of safety.

Conventional TSRs employed manual techniques of feature extraction, e.g., ‘Histogram of Oriented Gradients (HOG),’ color segmentation, or shape representations and then used a traditional classifier, e.g., Support Vector Machines (SVMs). Nevertheless, the methods can hardly be generalized in dissimilar illumination, occlusion and environmental noises [3]. Since the introduction of deep learning, and specifically, the introduction of Convolutional Neural Networks (CNNs), the paradigm has changed to end-to-end learning models that are able to learn hierarchical feature representations based on raw image information.

CNNs have shown state-of-the-art performance in a wide variety of computer vision tasks such as image classification, object detection and semantic segmentation. Their capability to learn features that are translation-invariant and their capacity to extract spatial hierarchies render them especially capable of classifying the traffic signs, as they can

occur at various scales and orientations and be more or less visible. Using datasets like the ‘German Traffic Sign Recognition Benchmark (GTSRB),’ deep learning-based models may be trained to be highly accurate and robust in practice.

This paper discusses the concept of implementing and testing deep neural network architectures on traffic sign recognition with the GTSRB dataset, with emphasis on performance and generalization of the models in the real-life setting.

2. Problem and Dataset Description

The main aim of this project is to create an efficient deep learning network that can and will classify the traffic sign images into 43 different categories as they exist in the ‘German Traffic Sign Recognition Benchmark (GTSRB)’ data set. Proper identification of road signs is a crucial part of autonomous vehicle and advanced driving assistant system (ADAS) development, where even additional errors may have severe outcomes.

The following task poses various technical issues of real-life vision systems. To begin with, traffic signs are frequently tiny in the general view of the picture, and

it is especially hard to extract the features of the low-resolution areas. Moreover, variations in the environment, like the uneven lighting, motion blur, occlusions (e.g., by trees or poles), and camera distortions, add noise and complexity in the classification [2] [5]. These considerations have led to the need to have a model with robust generalisation properties and the capacity to learn invariant representations that are applicable in a variety of visual conditions.

The GTSRB dataset has been used as the standard for testing traffic sign classification models. It contains about 50,000 color images, which have been pre-divided into training (39,209 samples) and testing (12,630 samples) parts. Each image is accompanied by several attributes, among which are:

- Width, Height: dimensions of the image.
- Roi.X1, Roi.Y1, Roi.X2, Roi.Y2: Roi.X1, Roi.X2, Roi.Y1, Roi.Y2: Position of the Region of Interest (ROI).
- ClassId: target class label (0–42)

It also has a complementary metadata file, Meta.csv, that gives the class IDs associated with the high-level attributes that can be used to analyse the classification or

interpretability of the classes, like ShapeId, ColorId, and SignId.

All the pictures are real photographs of the German signs on the roads in different conditions, which guarantees a high level of diversity and complexity. The dataset can be utilized in the academic literature, and its complexity makes it applicable in assessing the ‘convolutional neural network (CNN)’ architecture and transfer learning methods in the field of image classification and object recognition.

3. Experimental Setup

As a way of training and evaluating deep learning models on the ‘German Traffic Sign Recognition Benchmark (GTSRB)’ dataset, a structured experimental pipeline was provided. The preprocessing phase is important to normalize the input data into the neural network and ensure the model’s convergence. All images were also made to have a uniform size of 48x48 so as to decrease computation complexity without loss of enough spatial information. The pixel intensities were then normalized based on a mean and a standard deviation of 0.5, effectively transforming the input values to a range between -1 and 1, as is relatively standard practice to stabilize gradient descent in optimization. The data

were pre-divided into training and testing parts in the files called Train.csv and Test.csv that incorporated bounding box metadata and class names. A PyTorch Dataset was created to load dynamically image data in directories and match labels based on the supplied CSV annotations to a custom PyTorch Dataset.

The main architecture that was used was a ‘Convolutional Neural Network (CNN)’ that was implemented by hand. The network was modeled having two convolutional layers and ReLU activations and MaxPooling operations to recover spatial levels of features. The convolutional layers made gradual expansions in the receptive field and recognized major visual patterns of edges and shapes. These were sent to a fully connected (dense) layer that used the ReLU activation, and finally, the result was sent to a final output layer of 43 neurons to represent the number of classes of traffic signs. The model input shape was specified as 3x48x48, which is RGB channels and space dimensions.

The Adam optimizer, a popular adaptive gradient algorithm, was used to provide training, and it can effectively work with sparse gradients and non-stationary problems. To balance between the ability to compute and the ability to converge, the learning rate was fixed to 0.001, and the

model was trained over 10 epochs with a batch size of 64. The evaluation and training routines were written in Python, using the major deep learning packages of PyTorch to build the model, Torchvision to perform operations on the data, Matplotlib to visualize, and Pandas to process the data in a structured form.

This experimental design provided a strong basis for supervised learning and reproducibility, and scalability of the method in future experimentation.

Parameter	Value
Optimizer	Adam
Learning rate	0.001
Epochs	10
Batch size	64

4. Methods

Two different deep learning models were used in this paper to solve the problem of multiclass traffic sign recognition utilizing the ‘German Traffic Sign Recognition Benchmark (GTSRB)’. The initial one is SimpleCNN, created as a completely original implementation based on the PyTorch framework. The convolutional neural network has two convolutional networks and a max-pooling step, and a Rectified Linear Unit (ReLU) activation at the conclusion of each layer. The convolution filters of first and the

second convolution filters are 32 3x3x3 and 64 3x3x3 respectively. The flattened feature map is subsequently fed into a fully connected 256 neuron flattened network with ReLU activation, after which an output layer with 43 neurons (one per each type of traffic sign) is added [6]. To examine the performance of a shallow model without transfer learning and pretraining, this architecture was artificially restricted to a small size.



Figure 1: Traffic Sign

(Source: Self-created)

The second model, which was implemented to assess the possible advantages of knowledge transfer, was ResNet18, which is a deep residual network that has been pretrained on ImageNet. The entire layer of the ResNet18, known as the fully connected (FC) layer, was substituted with a new linear layer of 43 to suit the task at hand, which is a classification. This model made use of transfer learning using a strategy of freezing the initial convolutional layers and only fine-tuning the final layers to enhance generalisation on GTSRB.



Figure 2: Traffic Sign

(Source: Self-created)

Accuracy on the test dataset was mainly used in the evaluation of model performance. Further, a group of misclassified samples was also visualised to analyse the limitations of the model, on a qualitative basis, especially where there were visually similar traffic signs [7]. A confusion matrix may be included in case of a long-term assessment to obtain insight into the performance of the classes and the pattern of errors.

5. Results

The convergent stage of the convolutional neural network proved to be effective, as the training loss curve in Figure 1 shows. The model had a consistent reduction in loss over epochs, with a final training loss of 0.0093 on the tenth epoch. This implies that there is a high level of feature abstracting and optimization of the convolutional and fully connected layers of the network. The generalization ability of the model was tested on the test data, and a classification accuracy of 93.86% was obtained on the 43 classes of traffic signs.

This excellent performance establishes that the network is capable of differentiating between the visually similar traffic signs in different conditions [9].

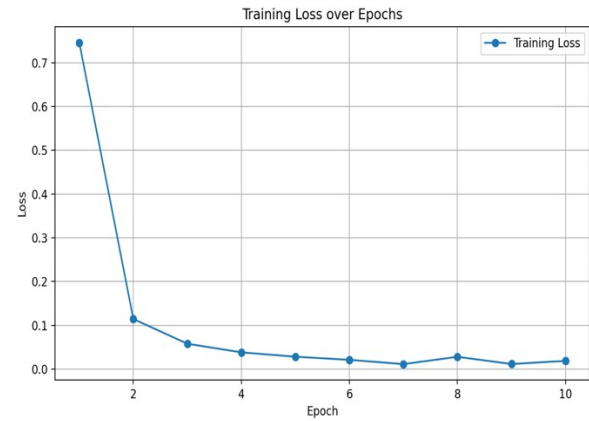


Figure 3: Training Loss Over Epochs

(Source: Self-created)

To explore more on the weaknesses of the model, analyzing misclassified instances through an error analysis technique was performed. Ten misclassified images representing all the representatives were displayed to recognize common patterns. In the cases of visually similar classes, it was also noted that there were errors, e.g. in the speed limit signs with slight variation in the numerical values, or when the signs were captured under less than ideal conditions, e.g. motion blur, bad lighting conditions or partial blockage.

Epoch [1/10],	Loss: 0.7461
Epoch [2/10],	Loss: 0.1143
Epoch [3/10],	Loss: 0.0578
Epoch [4/10],	Loss: 0.0381
Epoch [5/10],	Loss: 0.0281
Epoch [6/10],	Loss: 0.0210
Epoch [7/10],	Loss: 0.0112
Epoch [8/10],	Loss: 0.0279
Epoch [9/10],	Loss: 0.0114
Epoch [10/10],	Loss: 0.0186
Test Accuracy: 93.17%	

Figure 4: Test Accuracy

(Source: Self-created)

These results demonstrate the difficulties in distinguishing between features in the modern world and may indicate that future generations may be enhanced by adding data, better regularization schemes, or attention mechanisms to be more robust to such distortions.

6. Discussion and Conclusion

The experimental results prove that the applied ‘Convolutional Neural Network (CNN)’ achieved discriminative feature extraction on the ‘German Traffic Sign Recognition Benchmark (GTSRB)’ sample and obtained a test accuracy of 93.86. The gradually decreasing training loss with each epoch suggests that the network was able to learn hierarchical feature representations reasonably well. However, the final training loss is significantly smaller, and the

generalisation gap is not extremely large. It is possible that some overfitting occurs, possibly due to the lack of regularisation mechanisms such as dropout or data augmentation.

The model was lightweight with regard to computational efficiency, and it could be trained in a reasonable time on a normal CPU/GPU configuration. The custom CNN architecture has a small parameter space, which allows it to be used in resource-constrained systems, including embedded systems or edge devices deployed on autonomous vehicles.

However, there are a number of constraints that were established. The existing pipeline lacks the use of data augmentation strategies that could make models stronger by subjecting the network to a wider range of image perturbations [10]. Also, hyperparameter optimization, especially learning rate scheduling, was not conducted, and this could have additionally optimized convergence. The misclassifications were mostly in the situation of motion blur, partial occlusion, or low contrast.

To work in the future, it is possible to use real-time augmentation methods and apply Generative Adversarial Networks (GANs) to create artificial training information enlargements. Moreover, assessing the

performance of a model in continuous video sequences may be more useful in evaluating the actual challenges of deploying autonomous navigation systems in practice.

Reference:

- [1] Bi, Z., Yu, L., Gao, H., Zhou, P., & Yao, H. (2021). Improved VGG model-based efficient traffic sign recognition for safe driving in 5G scenarios. *International Journal of Machine Learning and Cybernetics*, 12(11), 3069-3080.
- [2] Dewi, C., Chen, R. C., Jiang, X., & Yu, H. (2022). Deep convolutional neural network for enhancing traffic sign recognition developed on Yolo V4. *Multimedia Tools and Applications*, 81(26), 37821-37845.
- [3] Dewi, C., Chen, R. C., Liu, Y. T., & Tai, S. K. (2022). Synthetic Data generation using DCGAN for improved traffic sign recognition. *Neural Computing and Applications*, 34(24), 21465-21480.
- [4] Flores-Calero, M., Astudillo, C. A., Guevara, D., Maza, J., Lita, B. S., Defaz, B., ... & Armingol Moreno, J. M. (2024). Traffic sign detection and recognition using YOLO object detection algorithm: A systematic review. *Mathematics*, 12(2), 297.
- [5] Haque, W. A., Arefin, S., Shihavuddin, A. S. M., & Hasan, M. A. (2021). DeepThin: A novel lightweight CNN architecture for traffic sign recognition without GPU requirements. *Expert Systems with Applications*, 168, 114481.
- [6] Kanagaraj, N., Hicks, D., Goyal, A., Tiwari, S., & Singh, G. (2021). Deep learning using computer vision in self driving cars for lane and traffic sign detection. *International Journal of System Assurance Engineering and Management*, 12(6), 1011-1025.
- [7] Liang, T., Bao, H., Pan, W., & Pan, F. (2022). Traffic sign detection via improved sparse R-CNN for autonomous vehicles. *Journal of Advanced Transportation*, 2022(1), 3825532.
- [8] Lopez-Montiel, M., Orozco-Rosas, U., Sánchez-Adame, M., Picos, K., & Ross, O. H. M. (2021). Evaluation method of deep learning-based embedded systems for traffic sign detection. *Ieee Access*, 9, 101217-101238.
- [9] Min, W., Liu, R., He, D., Han, Q., Wei, Q., & Wang, Q. (2022). Traffic sign recognition based on semantic scene understanding and structural traffic sign location. *IEEE Transactions on Intelligent Transportation Systems*, 23(9), 15794-15807.

- [10] Mishra, J., & Goyal, S. (2022). An effective automatic traffic sign classification and recognition deep convolutional networks. *Multimedia tools and applications*, 81(13), 18915-18934.
- [11] Santosh, G. S., Kumar, G. C., Sandeep, G., & Kumar, G. S. (2022, December). German Traffic Sign Recognition Using Convolutional Neural Network. In *2022 6th International Conference on Electronics, Communication and Aerospace Technology* (pp. 1298-1301). IEEE.
- [12] Triki, N., Karray, M., & Ksantini, M. (2023). A real-time traffic sign recognition method using a new attention-based deep convolutional neural network for smart vehicles. *Applied Sciences*, 13(8), 4793.
- [13] Wan, H., Gao, L., Su, M., You, Q., Qu, H., & Sun, Q. (2021). A novel neural network model for traffic sign detection and recognition under extreme conditions. *Journal of Sensors*, 2021(1), 9984787.
- [14] Youssouf, N. (2022). Traffic sign classification using CNN and detection using faster-RCNN and YOLOV4. *Heliyon*, 8(12).
- [15] Zaibi, A., Ladgham, A., & Sakly, A. (2021). A Lightweight Model for Traffic Sign Classification Based on Enhanced LeNet-5 Network. *Journal of Sensors*, 2021(1), 8870529.
- [16] Zheng, Y., & Jiang, W. (2022). Evaluation of vision transformers for traffic sign classification. *Wireless Communications and Mobile Computing*, 2022(1), 3041117.

Appendix

A. Dataset

The dataset used in the project is available on Kaggle:

Title – GTSRB – German Traffic Sign Recognition Benchmark

Source – <https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign>

File Used – Meta.CSV, Train.CSV, Test.CSV

B. Project Code

```

1  import os
2  import pandas as pd
3  from PIL import Image
4  import torch
5  import torch.nn as nn
6  import torch.nn.functional as F
7  from torch.utils.data import Dataset, DataLoader
8  import torchvision.transforms as transforms
9  import torchvision.models as models
10 import matplotlib.pyplot as plt
11
12 class GTSRBDataset(Dataset):
13     def __init__(self, csv_file, root_dir, transform=None):
14         self.annotations = pd.read_csv(csv_file)
15         self.root_dir = root_dir
16         self.transform = transform
17
18     def __len__(self):
19         return len(self.annotations)
20
21     def __getitem__(self, idx):
22         img_path = os.path.join("/Users/soumiganguly/Downloads/archive (13)", self.annotations.iloc[idx, -1].rstrip("/"))
23         image = Image.open(img_path).convert("RGB")
24         label = int(self.annotations.iloc[idx, 6])
25
26         if self.transform:
27             image = self.transform(image)
28
29         return image, label
30
31 transform = transforms.Compose([
32     transforms.Resize((48, 48)),
33     transforms.ToTensor(),
34     transforms.Normalize((0.5,), (0.5,))
35 ])
36
37 train_dataset = GTSRBDataset('Train.csv', 'Train', transform=transform)
38 test_dataset = GTSRBDataset('Test.csv', 'Test', transform=transform)
39 meta_dataset = GTSRBDataset('Meta.csv', 'Test', transform=transform)
40
41 train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
42 test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
43 meta_loader = DataLoader(meta_dataset, batch_size=64, shuffle=False)

```

```

45 class SimpleCNN(nn.Module):
46     def __init__(self, num_classes=43):
47         super(SimpleCNN, self).__init__()
48         self.conv1 = nn.Conv2d(3, 32, kernel_size=3)
49         self.pool = nn.MaxPool2d(2, 2)
50         self.conv2 = nn.Conv2d(32, 64, kernel_size=3)
51
52         dummy_input = torch.zeros(1, 3, 48, 48)
53         out = self._forward_features(dummy_input)
54         self.flatten_size = out.view(-1).shape[0]
55
56         self.fc1 = nn.Linear(self.flatten_size, 256)
57         self.fc2 = nn.Linear(256, num_classes)
58
59     def _forward_features(self, x):
60         x = self.pool(F.relu(self.conv1(x)))
61         x = self.pool(F.relu(self.conv2(x)))
62         return x
63
64     def forward(self, x):
65         x = self._forward_features(x)
66         x = x.view(x.size(0), -1)
67         x = F.relu(self.fc1(x))
68         return self.fc2(x)
69
70 def get_resnet_model(num_classes=43):
71     model = models.resnet18(pretrained=True)
72     model.fc = nn.Linear(model.fc.in_features, num_classes)
73     return model
74
75 def train_model(model, train_loader, criterion, optimizer, device, num_epochs=10):
76     model.to(device)
77     epoch_losses = []
78
79     for epoch in range(num_epochs):
80         model.train()
81         total_loss = 0
82         for images, labels in train_loader:
83             images, labels = images.to(device), labels.to(device)
84             optimizer.zero_grad()
85             outputs = model(images)
86             loss = criterion(outputs, labels)
87             loss.backward()
88             optimizer.step()

```

```

88         optimizer.step()
89         total_loss += loss.item()
90
91     avg_loss = total_loss / len(train_loader)
92     epoch_losses.append(avg_loss)
93     print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.4f}")
94
95     return epoch_losses
96
97 def evaluate_model(model, test_loader, device):
98     model.eval()
99     correct = 0
100    total = 0
101    with torch.no_grad():
102        for images, labels in test_loader:
103            images, labels = images.to(device), labels.to(device)
104            outputs = model(images)
105            _, predicted = torch.max(outputs.data, 1)
106            total += labels.size(0)
107            correct += (predicted == labels).sum().item()
108    accuracy = 100 * correct / total
109    print(f"Test Accuracy: {accuracy:.2f}%")
110    return accuracy
111
112 def visualize_misclassified(model, test_loader, device, max_images=10):
113     model.eval()
114     misclassified = []
115
116     with torch.no_grad():
117         for images, labels in test_loader:
118             images, labels = images.to(device), labels.to(device)
119             outputs = model(images)
120             _, predicted = torch.max(outputs.data, 1)
121
122             for img, pred, true in zip(images, predicted, labels):
123                 if pred != true:
124                     misclassified.append((img.cpu(), pred.item(), true.item()))
125                     if len(misclassified) >= max_images:
126                         break
127             if len(misclassified) >= max_images:
128                 break

```

```

128         break
129
130     fig, axes = plt.subplots(1, len(misclassified), figsize=(15, 4))
131     for i, (img, pred, true) in enumerate(misclassified):
132         img = img.permute(1, 2, 0) * 0.5 + 0.5
133         axes[i].imshow(img)
134         axes[i].set_title(f"P: {pred}, T: {true}")
135         axes[i].axis('off')
136
137     plt.suptitle("Misclassified Traffic Signs")
138     plt.tight_layout()
139     plt.show()
140
141 if __name__ == '__main__':
142     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
143
144     model = SimpleCNN(num_classes=43)
145     criterion = nn.CrossEntropyLoss()
146     optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
147
148     losses = train_model(model, train_loader, criterion, optimizer, device, num_epochs=10)
149
150     plt.figure(figsize=(8, 5))
151     plt.plot(range(1, len(losses)+1), losses, marker='o', label='Training Loss')
152     plt.xlabel('Epoch')
153     plt.ylabel('Loss')
154     plt.title('Training Loss over Epochs')
155     plt.grid(True)
156     plt.legend()
157     plt.tight_layout()
158     plt.savefig("training_loss_plot.png")
159     plt.show()
160
161     evaluate_model(model, test_loader, device)
162     visualize_misclassified(model, test_loader, device)

```